

Salesforce CRM to Extole (Apex and Flows)

Send event data from Salesforce CRM (Salesforce Sales Cloud) to Extole using Apex triggers and automate your processes using Flows.

Overview

Integrating Extole and Salesforce CRM (also known as Salesforce Sales Cloud) allows you to set up Apex triggers that correspond to key events in your lead generation program design. When these events occur, you can configure Flows so that the data automatically fires into Extole.

Prerequisites

Requirements	Description
Salesforce Sales Cloud Account and Admin Access	You must have a Salesforce Sales Cloud account and admin-level access, as well as familiarity with the platform, in order to set up this integration.
Extole Account	You must have an Extole account to leverage this integration.

Integration

Send Lead Created Events

When leads are created, trigger a RESTful API call to Extole.

Set Up Basic Configuration

You must be a system administrator in order to complete the following steps.

1. Navigate to Setup > Remote Site Settings > New and add `<https://api.extole.io >`.
2. Go to Setup > Apex Classes > New and create an Apex class with an invocable method by copying and pasting the code below.
3. Edit the code to query fields on your specific Lead object (line 9), and add those in to be sent to Extole in the request body (line 12)

```

public with sharing class ExtoleLeadController{
    @InvocableMethod
    public static void getLeadIds(List<String>leadIds){
        makeCallOut(leadIds);
    }

    @future(callout=true)
    public static void makeCallOut(List<String>leadIdList){
        List<Lead> leadRec = [SELECT Id LastName,FirstName,Email,Company,AdvocateCode FROM Lead WHERE
        Id IN:leadIdList];

        for(Lead l:leadRec){
            String finalBody='{"event_name":"lead_created","data":{"'+
                '"lead_id":"' +l.Id+'",' +
                '"first_name":"' +l.FirstName+'",' +
                '"last_name":"' +l.LastName+'",' +
                '"email":"' +l.Email+'",' +
                '"company":"' +l.Company+'",' +
                '"advocate_code":"' +l.AdvocateCode+'', "app_type":"salesforce_crm"}}';

            String endPoint = 'https://api.extole.io/v5/events';
            HttpRequest http = new HttpRequest();
            http.setEndpoint(endPoint);
            http.setMethod('POST');
            http.setBody(finalBody);
            http.setHeader('Content-Type', 'application/json');
            http.setHeader('Authorization', 'Bearer <INSERT TOKEN HERE>');
            Http h = new Http();
            HttpResponse response = h.send(http);
        }
    }
}

```

Automate using Flow on Lead Object

1. Go to Setup > Process Automation > Flow > New Flow > Record Triggered Flow > Create.
2. Select the Object and configure the Trigger by checking the option A record is created .
3. Choose to optimize the flow for Actions and Related Records.

Configure Start

Select Object

Select the object whose records trigger the flow when they're created, updated, or deleted.

* Object

Lead

Configure Trigger

* Trigger the Flow When:

- ☒ A record is created
- ☐ A record is updated
- ☐ A record is created or updated
- ☐ A record is deleted

Set Entry Conditions

Specify entry conditions to reduce the number of records that trigger the flow and the number of times the flow is executed. Minimizing unnecessary flow executions helps to conserve your org's resources.

If you create a flow that's triggered when a record is updated, we recommend first defining entry conditions. Then select the **Only when a record is updated to meet the condition requirements** option for When to Run the Flow for Updated Records.

Condition Requirements

None

New

Formulas for Conditions ⓘ

* Optimize the Flow for:

Fast Field Updates

Update fields on the record that triggers the flow to run. This high-performance flow runs *before* the record is saved to the database.

Actions and Related Records

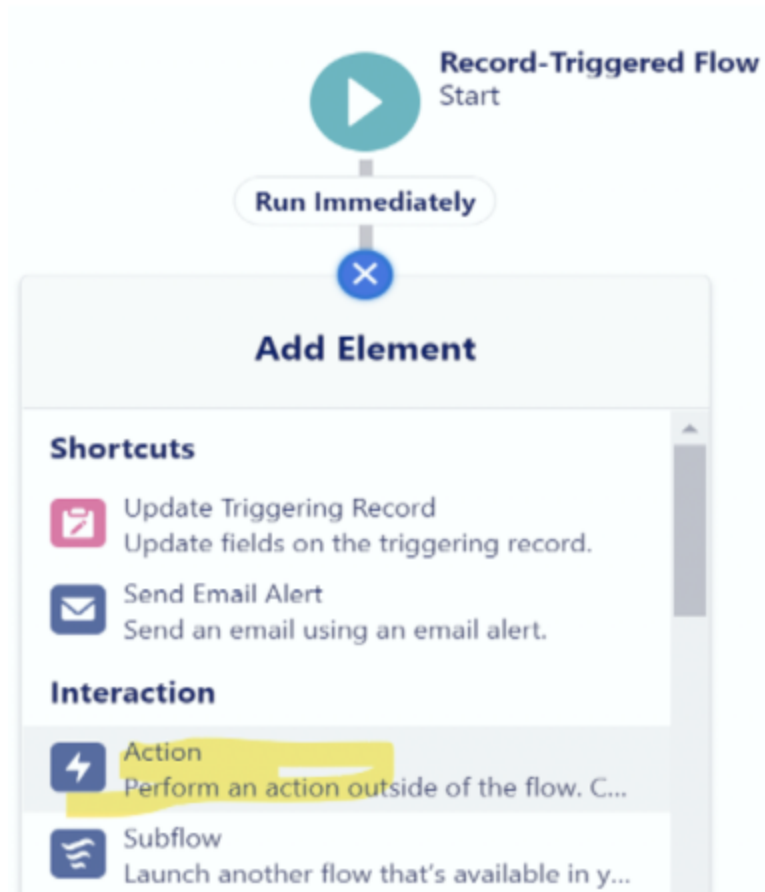
Update any record and perform actions, like send an email. This more flexible flow runs *after* the record is saved to the database.

- ☐ Include a Run Asynchronously path to access an external system after the original transaction for the triggering record is successfully committed

Cancel

Done

4. Click on the + sign to add an element, then select **Action**.



5. Select the Apex Action and Controller you just created.

New Action

Filter By

Type

Core Action

Apex Action

Apex Action (Legacy)

Email Alert

Action

Extol



Lights, camera, action!

Select an Apex action to configure.

Cancel

Done

5. Select Apex Action and set the input value for `leadId` as `{!$Record.Id}`.

Edit "ExtoleLeadController" Apex action

Use values from earlier in the flow to set the inputs for the "ExtoleLeadController" Apex action. To use its outputs later in the flow, store them in variables.

Extole Controller (Extole_Controller) 

Set Input Values

A_a

leadId



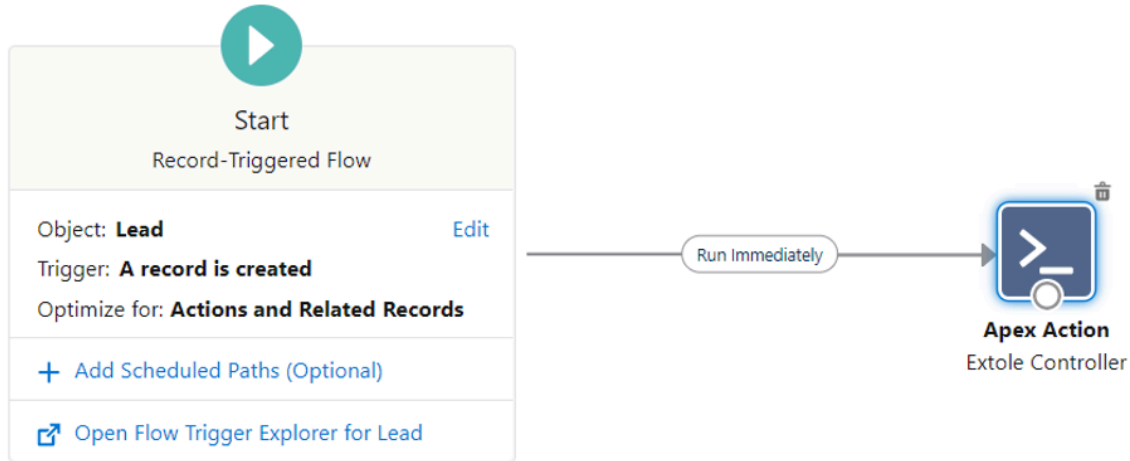
Include

> Advanced

Cancel

Done

6. Your final result should look like this:



7. Activate the flow and test it by creating the Lead Record.

Send Lead Converted (Opportunity Created) Events

When created leads convert to an opportunity, trigger a RESTful API call to Extol.

Set Up Basic Configuration

You must be a system administrator in order to complete the following steps. The flow will be triggered when the lead will be converted (not on opportunity creation).

1. Navigate to Setup > Remote Site Settings > New and add `<https://api.extole.io >`.
2. Go to Setup > Apex Classes > New and create a new Apex class with an invocable method by copying and pasting the code below.
3. Edit the code to query fields on your specific Lead object (line 9), and add those in to be sent to Extol in the request body (line 12)

```
java
```

```

public with sharing class ExtoleConvertedLeadController{
    @InvocableMethod
    public static void getLeadIds(List<String>leadIds){
        makeCallOutForConvertedLeads(leadIds);
    }

    @future(callout=true)
    public static void makeCallOutForConvertedLeads(List<String>leadIdList){
        List<Lead>leadRec = [SELECT Id,LastName,FirstName,Email,AdvocateCode FROM Lead WHERE Id
IN:leadIdList];

        for(Lead l:leadRec){
            String finalBody='{"event_name":"lead_converted","data":{"'+
                '"lead_id":"' +l.Id+'",'+
                '"first_name":"' +l.FirstName+'",'+
                '"last_name":"' +l.LastName+'",'+
                '"email":"' +l.Email+'",'+
                '"advocate_code":"' +l.AdvocateCode+', "app_type":"salesforce_crm"}}';

            String endPoint = 'https://api.extole.io/v5/events';
            HttpRequest http = new HttpRequest();
            http.setEndpoint(endPoint);
            http.setMethod('POST');
            http.setBody(finalBody);
            http.setHeader('Content-Type', 'application/json');
            http.setHeader('Authorization', 'Bearer <INSERT TOKEN HERE>');
            Http h = new Http();
            HttpResponse response = h.send(http);
        }
    }
}

```



Important Notes

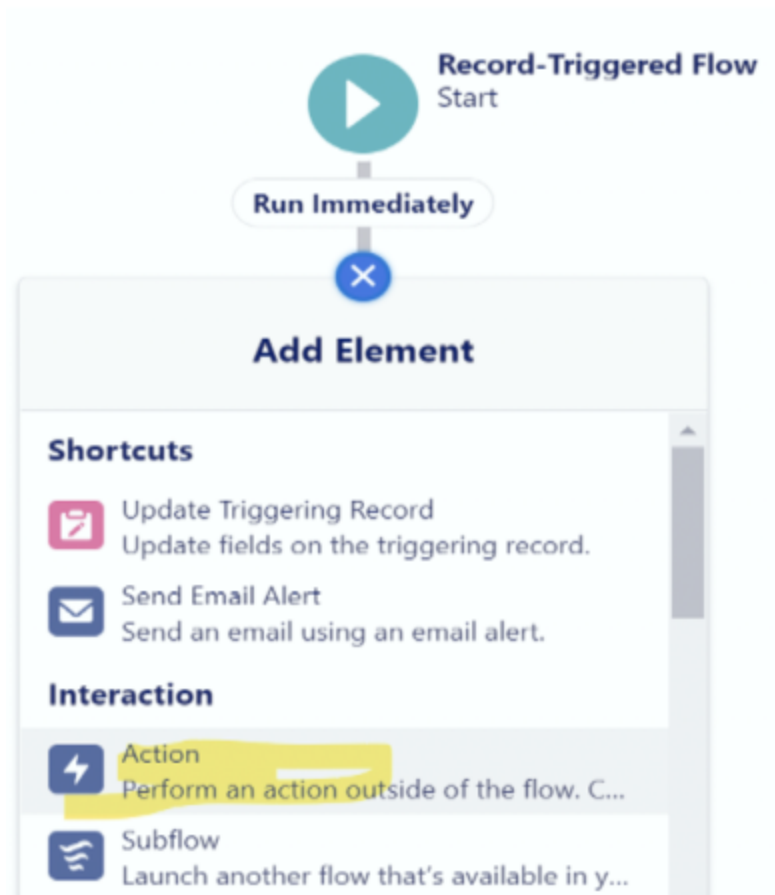
- This sample code may not reflect your SFDC configuration or custom fields. It may need updating to use fields your team is taking advantage of.
- Feel free to rename the label or variable as per your org coding guidelines.
- The method is tagged with `@InvocableMethod` , which enables this class to be used by Flow.
- Keep the auth token in custom metadata or a custom label.

Automate using Flow on Lead Object

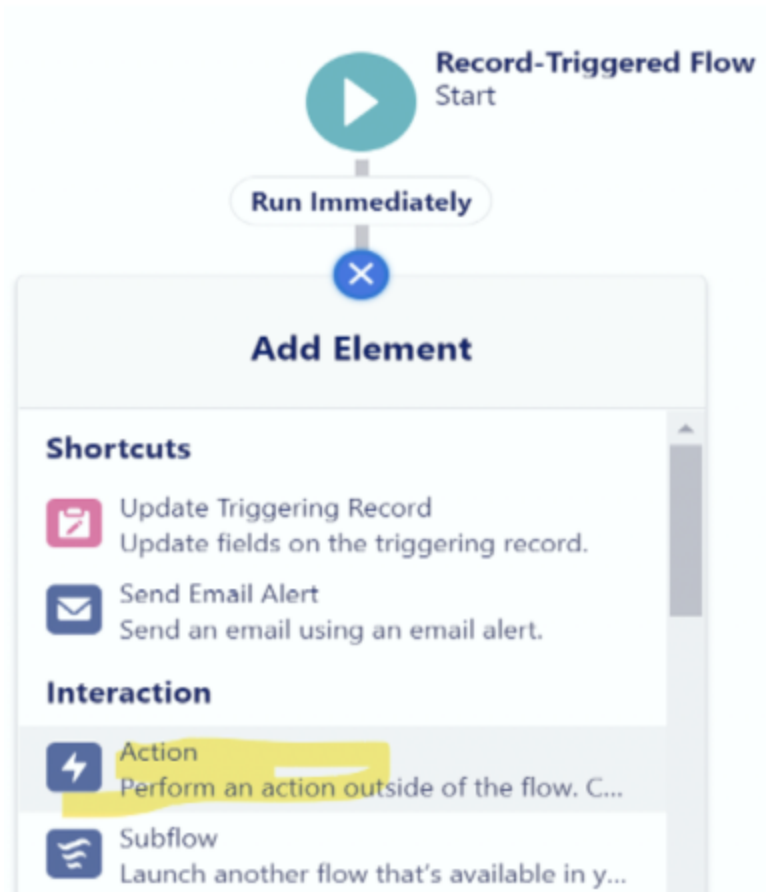
For the following steps, you must have System Administrator permission. You must also have a basic understanding of how to create a flow and add filters and actions.

1. Go to Setup > Process Automation > Flow > New Flow > Record Triggered Flow > Create.
2. Select the Object and configure the Trigger by checking the option `A record is updated` .
3. Open the Condition Requirements dropdown and select `All Conditions Are Met (AND)` . Set the Field as `IsConverted` , the Operator as `Equals` , and the Value as `True` .
4. Choose to run the flow for updated records every time a record is **updated to meet the condition requirements**.

5. Choose to optimize the flow for Actions and Related Records.



6. Click on the + sign to add an element, then select Action .



7. Select the Apex Action and Controller you just created.

New Action

Filter By
Type ▼

Core Action
Apex Action
Apex Action (Legacy)
Email Alert

Action
Extole



Lights, camera, action!

Select an Apex action to configure.

Cancel Done

8. Select the Apex Action and set the input value for `leadId` as `{!$Record.Id}` .

Edit "ExtoleConvertedLeadController" Apex action

Use values from earlier in the flow to set the inputs for the "ExtoleConvertedLeadController" Apex action. To use its outputs later in the flow, store them in variables.

Extole Converted Lead (Extole_Converted_Lead) 

Set Input Values

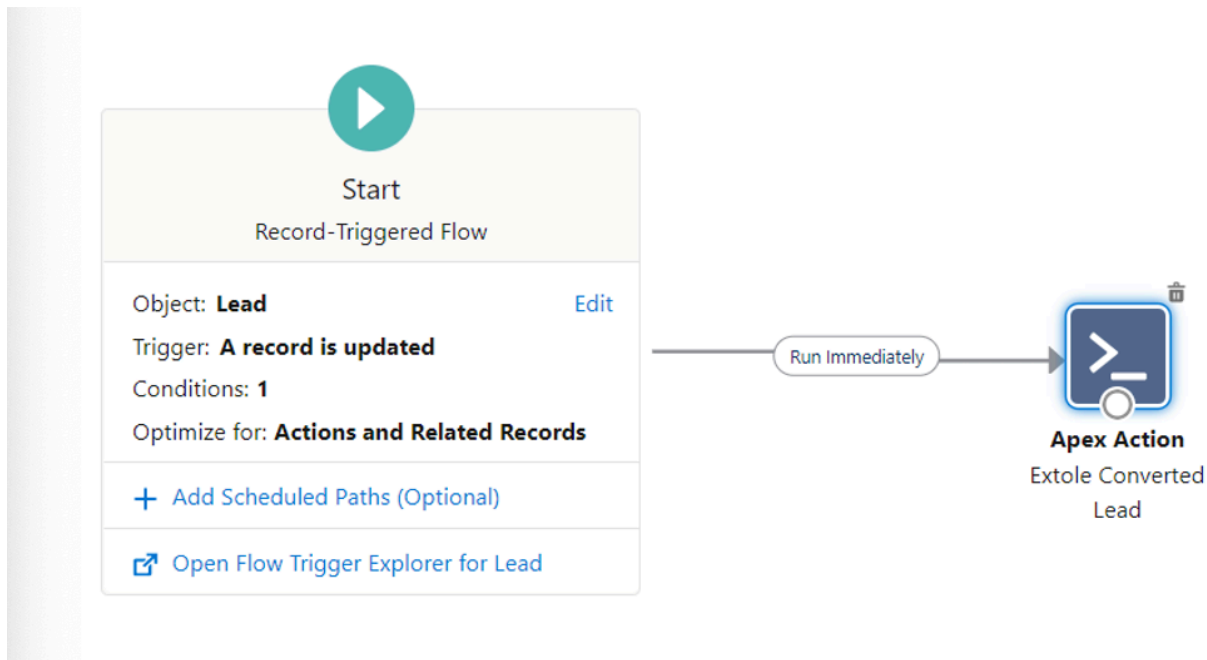
A_a leadId

{!\$Record.Id}

☒ Include

Cancel Done

9. Your final result should look like this:



10. Active the flow and test it by converting the Lead Record.

Send Opportunity Closed / Won Events

When an opportunity is closed or won, trigger a RESTful API call to Extole.

Set Up Basic Configuration

You must be a system administrator in order to complete the following steps.

1. Navigate to Setup > Remote Site Settings > New and add `<https://api.extole.io >`.
2. Go to Setup > Apex Classes > New and create a new Apex class with an invocable method by copying and pasting the following code:
3. Edit the code to query fields on your specific Opportunity object (line 9), and add those in to be sent to Extole in the request body (line 12). You may need to use an identifier in the Opportunity object to query a different Object to get the required data. This should happen inside of the for loop on line 11

JSON

```

public with sharing class ExtoleOpportunityController{
    @InvocableMethod
    public static void getOppoIds(List<String>opportunityId){
        makeCallOutForWonOpportunity(opportunityId);
    }

    @future(callout=true)
    public static void makeCallOutForWonOpportunity(List<String>opportunityIdList){
        List<Opportunity>oppRec = [SELECT Id,Name,Email FROM Opportunity WHERE Id
        IN:opportunityIdList];

        for(Opportunity op:oppRec){
            String finalBody='{"event_name":"opportunity_closedwon","data":{"lead_id":"123LeadId",' +
                '"opportunity_id":"' +op.Id+'",' +
                '"first_name":"' +op.Name+'",' +
                '"email":"' +op.Email+'","app_type":"salesforce_crm"}}';

            String endPoint = 'https://api.extole.io/v5/events';
            HttpRequest http = new HttpRequest();
            http.setEndpoint(endPoint);
            http.setMethod('POST');
            http.setBody(finalBody);
            http.setHeader('Content-Type', 'application/json');
            http.setHeader('Authorization', 'Bearer <INSERT TOKEN HERE>');
            Http h = new Http();
            HttpResponse response = h.send(http);
        }
    }
}

```



Important Notes

- This sample code may not reflect your SFDC configuration or custom fields. It may need updating to use fields or objects your team is taking advantage of.
- Feel free to rename the label or variable as per your org coding guidelines.
- The method is tagged with `@InvocableMethod` , which enables this class to be used by Flow.
- Keep the auth token in custom metadata or a custom label.

Automate using Flow on Lead Object

For the following steps, you must have System Administrator permission. You must also have a basic understanding of how to create a flow and add filters and actions.

1. Go to Setup > Process Automation > Flow > New Flow > Record Triggered Flow > Create.
2. Select the Object and configure the Trigger by checking the option A record is created or updated .
3. Open the Condition Requirements dropdown and select All Conditions Are Met (AND) . Set the Field as StageName , the Operator as Equals , and the Value as Closed Won .
4. Choose to run the flow for updated records every time a record is **updated to meet the condition requirements**.

5. Choose to optimize the flow for Actions and Related Records.

Configure Start

Select Object

Select the object whose records trigger the flow when they're created, updated, or deleted.

* Object

Opportunity

Configure Trigger

* Trigger the Flow When:

☐ A record is created

☐ A record is updated

☒ A record is created or updated

☐ A record is deleted

Set Entry Conditions

Specify entry conditions to reduce the number of records that trigger the flow and the number of times the flow is executed. Minimizing unnecessary flow executions helps to conserve your org's resources.

If you create a flow that's triggered when a record is updated, we recommend first defining entry conditions. Then select the **Only when a record is updated to meet the condition requirements** option for When to Run the Flow for Updated Records.

Condition Requirements

All Conditions Are Met (AND)

New

Formulas for Conditions ⓘ

Field

StageName

Operator

Equals

Value

Closed Won

+ Add Condition

When to Run the Flow for Updated Records ⓘ

☒ Every time a record is updated and meets the condition requirements

☐ Only when a record is updated to meet the condition requirements

* Optimize the Flow for:

Fast Field Updates

Update fields on the record that triggers the flow to run. This high-performance flow runs *before* the record is saved to the database.

Actions and Related Records

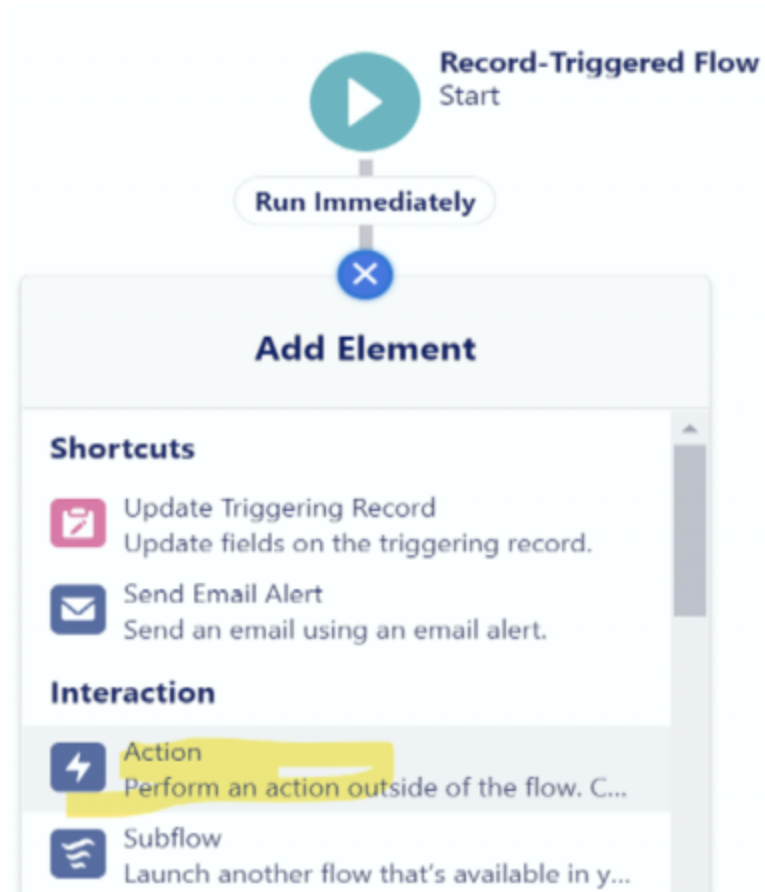
Update any record and perform actions, like send an email. This more flexible flow runs *after* the record is saved to the database.

☐ Include a Run Asynchronously path to access an external system after the original transaction for the triggering record is successfully committed

Cancel

Done

6. Click on the + sign to add an element, then select Action .



7. Select the Apex Action and Controller you just created.

New Action

Filter By

Type

Core Action

Apex Action

Apex Action (Legacy)

Email Alert

Action

Extole



Lights, camera, action!

Select an Apex action to configure.

Cancel

Done

8. Select the Apex Action and set the input value for opportunityId as {!\$Record.Id} .

Edit "ExtoleOpportunityController" Apex action

Use values from earlier in the flow to set the inputs for the "ExtoleOpportunityController" Apex action. To use its outputs later in the flow, store them in variables.

Oppo Closed Won (Oppo_Closed_Won)

Set Input Values

A_a

opportunityId

{!\$Record.Id}

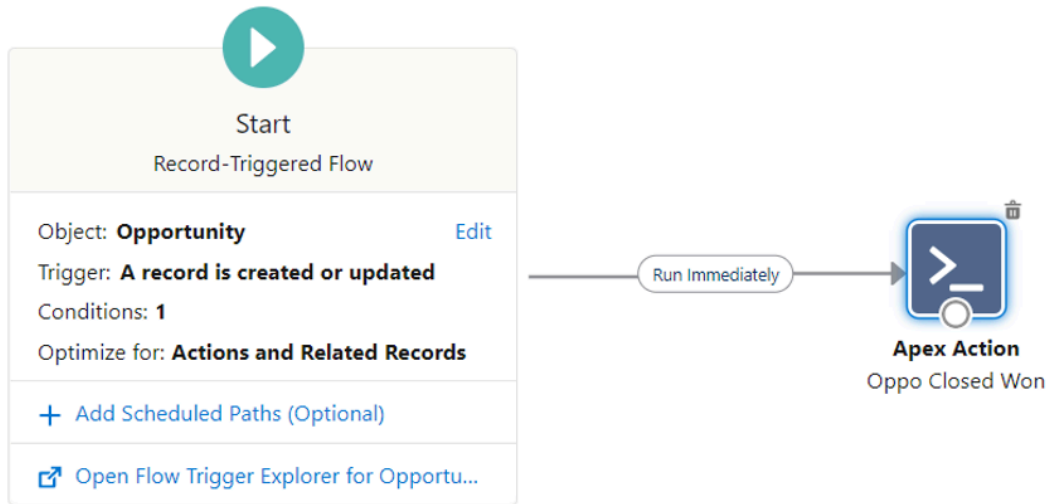
✓

Include

Cancel

Done

9. Your final result should look like this:



10. Activate and test the flow by changing or creating closed won opportunities.

How-To Guide

Debug Instructions

To debug, you can use the standard debugging options for Salesforce.

1. In your Apex code, put `System.debug()` where you would like to debug.
2. Go to Setup and type Debug Log into the search bar.
3. Click on Debug Logs from your search results.
4. Click the New button to enable the debug log for a user.
5. Select User from the Traced Entity Type dropdown.
6. Enter `SFDC_Developer` for the Traced Name Entity and Debug Level.
7. Save the log.



SETUP

Debug Logs

Configure your Debug Levels.

Cancel

Save

Traced Entity Type

User

Traced Entity Name

Start Date

23/08/2022 06:43

[23/08/2022 06:43]

Expiration Date

23/08/2022 07:13

[23/08/2022 06:43]

Debug Level

New Debug Level

Cancel

Save

8. After setting up the debug log, perform an operation to execute the flow/trigger. For example, you could create a lead, convert a lead, or close an opportunity.
9. Executing a flow will generate debug logs. Refresh the debug log page to see them.
10. Look for the most recent log and download it to see the response from Extol.



Debug Help

Still need help with debugging? Reference the following Salesforce articles.

- [Debugging Your Code](#)
- [Debugging Apex](#)

Send Additional Salesforce Events to Extol

There are two primary steps that must be set up for an event.

1. Create a Service Apex Class that can be used by a flow.
2. Create a flow to run for the specified events.

Create Service Apex Controller

You must be a system administrator in order to complete the following steps.

1. Navigate to Setup > Remote Site Settings > New and add <https://api.extole.io >.
2. Go to Setup > Apex Classes > New and create a new Apex class with an invocable method by copying and pasting the following code:

json

```
public with sharing class ExtoleController {
    @InvocableMethod
    public static void invokeFromFlow(List<String>recordIds){
        makeHttpCallToExtole(recordIds);
    }

    @future(callout=true)
    public static void makeHttpCallToExtole(List<String>recordIds){
        List<Opportunity>records = [SELECT Id,Name FROM <OBJECT NAME> WHERE Id IN: recordIds];

        for(OBJECT_NAME op: records){
            String jsonPayload = '{"<param 1>":"<value_1>","<param 2>":"<value_2>","app_type":"salesforce_crm"}';

            String endPoint = 'https://api.extole.io/v5/events';
            HttpRequest http = new HttpRequest();
            http.setEndpoint(endPoint);
            http.setMethod('POST');
            http.setBody(jsonPayload);
            http.setHeader('Content-Type', 'application/json');
            http.setHeader('Authorization', 'Bearer <INSERT TOKEN HERE>');
            Http h = new Http();
            HttpResponse response = h.send(http);
        }
    }
}
```



Important Notes

- Feel free to rename the label or variable as per your org coding guidelines.
- Replace the object name and update the query as necessary.
- Update the endpoint and other details as necessary.
- Keep the auth token in custom metadata or a custom label.

Create Flow

For the following steps, you must have System Administrator permission. You must also have a basic understanding of how to create a flow and add filters and actions.

1. Go to Setup > Process Automation > Flow > New Flow > Record Triggered Flow > Create.
2. Select the Object and configure the Trigger. For example, to fire the call when a lead is created, here you would select the Lead object.
3. Set the Entry Conditions according to your object and trigger.
4. Choose to optimize the flow for Actions and Related Records.
5. Click on the + sign to add an element, then select `Action`.
6. Select whatever Apex Action and controller you want.
7. Select Apex Action and set the input value as `{!$Record.Id}`.
8. Activate the flow and test it by conducting whatever action triggers the event.

Send Additional Data Attributes to Extole

Parameters are passed to Extole's Events API using Apex controllers. Parameters are passed a JSON (key, value pair). You can add as many additional data attributes as you want in the JSON.

 Updated 5 months ago

[← Hubspot](#)

[Extole to Salesforce CRM →](#)

Did this page help you?



Yes



No